

Team Members: Quan Do, Max Bennett, Ryan Vattuone

Course: CSEN 122L

Term: Spring 2026

CSEN 122 Project

I. Abstract

This project presents the design and implementation of a 32-bit pipelined CPU using Verilog HDL for the SCU Instruction Set Architecture (ISA). The processor supports the required SCU instruction set, including arithmetic, memory access, branching, and control flow operations. A pipelined datapath and control logic were developed to improve instruction throughput while keeping correct execution of the program. The CPU was modeled structurally in Verilog and verified through simulation using benchmark programs and waveform analysis. Functional testing was performed to ensure the correct execution of each instruction and proper operation of the pipeline. Performance analysis was conducted by evaluating instruction count, cycle count, CPI, and execution time for the benchmark program. The results demonstrate the successful implementation of a functional pipelined processor and provide insight into the performance advantages of pipelined execution.

II. SCU ISA Overview

The SCU ISA uses 32-bit instructions and a 64-register file. Each register is 32 bits wide and is named x0 through x63. The program counter is also 32 bits wide. Unlike RISC-V, this ISA uses word addressing, so the next sequential instruction is located at PC + 1 instead of PC + 4.

The standard instruction format is shown in Table 1. However, the SVPC instruction uses a special format, as shown in Table 2.

Table 1. SCU Instruction Format

Field	Bits	Description
opcode	[31:28]	Operation Code
rd	[27:22]	Destination Register
rs	[21:16]	Source Register 1
rt	[15:10]	Source Register 2
y	[9:0]	10-bit immediate

Table 2. SVPC Instruction Format

Field	Bits	Description
-------	------	-------------

opcode	[31:28]	Operation Code
rd	[27:22]	Destination Register
y	[21:0]	22-bit signed immediate

III. CPU Pipeline Design

The CPU was implemented as a five-stage pipeline:

IF → ID → EX → MEM → WB

The pipeline stages are separated using pipeline registers: IF/ID, ID/EX, EX/MEM, and MEM/WB. Each pipeline register stores the data and the control signals that are then used by later stages of the pipeline.

1. Instruction Fetch Stage

The Instruction Fetch stage is responsible for selecting the correct PC value and reading the instruction memory. Under normal execution, the PC simply increments by one because SCU uses word addressing. Therefore, the default next PC is:

$$PC_{next} = PC + 1$$

However, the PC may be redirected by either a branch or a jump-memory instruction. In the final design, BRZ and BRN are resolved in the ID stage, while JM is resolved later when the data memory output is available. Therefore, the PC redirect logic has to consider two possible redirect sources.

The branch redirect target is the value in R[rs], which is available in the ID stage. The JM redirect target is Mem[R[rs]], which is only available when the instruction reaches the memory stage. Therefore, the PC redirect decision can be described as:

```

if BranchTaken_ID:
    PC_next = BranchTarget_ID
else if EX/MEM.JM:
    PC_next = DataMemoryReadData
else:
    PC_next = PC + 1

```

Here, BranchTarget_ID is the value read from the register file for R[rs], and DataMemoryReadData is the data memory output used by the JM instruction. The instruction memory receives the current PC as its address and outputs the fetched instruction:

```
instruction_fetched = IM[PC]
```

After the instruction is fetched, the IF stage passes both the current PC and the fetched instruction into the IF/ID pipeline register. The IF/ID register stores:

```
IF/ID.PC  
IF/ID.Instruction
```

This register also has a flush input. A flush is needed when the CPU has already fetched an instruction from the wrong path. In the final design, the IF/ID register is flushed when either a branch is taken in ID or a JM redirect occurs later:

```
IF/ID_flush = BranchTaken_ID OR EX/MEM.JM
```

This prevents the wrongly fetched instruction from continuing into the pipeline.

2. Instruction Decode Stage

The Instruction Decode stage performs instruction decoding, register-file reading, immediate generation, control-signal generation, and branch decision logic. The instruction fields are extracted from the IF/ID instruction register:

```
opcode = IF/ID.Instruction[31:28]  
rd    = IF/ID.Instruction[27:22]  
rs    = IF/ID.Instruction[21:16]  
rt    = IF/ID.Instruction[15:10]
```

The immediate generator produces two possible immediate values. For most instructions, the 10-bit immediate field is sign-extended. For SVPC, the lower 22 bits are sign-extended. The selected immediate is:

```
if opcode = SVPC:  
    immediate = sign_extend(instruction[21:0])  
else:  
    immediate = sign_extend(instruction[9:0])
```

The register file reads two registers in the ID stage:

ReadData1 = R[rs]
ReadData2 = R[rt]

These two outputs are later used by ALU operations, memory address generation, store data, and branch target selection.

The control unit also operates in this stage. It reads the opcode and generates the control signals needed by the rest of the datapath, including RegWrite, MemRead, MemWrite, MemtoReg, ALUSrc, SVPC, JM, BRZ, BRN, and ALUOp.

In the final datapath, branch instructions are resolved in the ID stage. This means the CPU decides whether BRZ or BRN should branch before the instruction reaches EX. Since SCU branch instructions use the flags from the previous instruction, the branch decision is made using Z_prev and N_prev:

BRZ_taken = BRZ AND Z_prev
BRN_taken = BRN AND N_prev
BranchTaken_ID = BRZ_taken OR BRN_taken

If the branch is taken, the target address is the value in R[rs]. This gives:

BranchTarget_ID = R[rs]

A small branch-target forwarding check is included. If the branch target register is being written back in the same cycle, the CPU uses the writeback data instead of the older register-file output:

if MEM/WB.RegWrite AND MEM/WB.rd = ID.rs:
 BranchTarget_ID = WB_WriteData
else:
 BranchTarget_ID = R[rs]

The ID stage then passes all required values into the ID/EX register. This includes the control signals, the PC, both register-file outputs, the immediate value, and the register numbers. The ID/EX register is not flushed for a normal taken branch because the branch is already resolved in ID. At that point, only the instruction in IF/ID is wrong. However, ID/EX must be flushed when a JM redirect occurs, because JM is resolved later. Therefore:

ID/EX_flush = EX/MEM.JM

This is important because using the raw JM signal from ID to flush ID/EX would erase the JM instruction itself before it could reach memory.

3. Execute Stage

The Execute stage is responsible for selecting the ALU operands, applying forwarding, performing the ALU operation, and generating flags. The forwarding unit is needed because an instruction may depend on the result of a previous instruction that has not yet been written back to the register file.

The forwarding unit compares the source registers in the ID/EX stage with destination registers in EX/MEM and MEM/WB. For ALU input A, the logic is:

```
if EX/MEM.RegWrite AND EX/MEM.rd = ID/EX.rs:
```

```
    ForwardA = EX/MEM.ALUResult
```

```
else if MEM/WB.RegWrite AND MEM/WB.rd = ID/EX.rs:
```

```
    ForwardA = WB_WriteData
```

```
else:
```

```
    ForwardA = ID/EX.ReadData1
```

For ALU input B, the logic is:

```
if EX/MEM.RegWrite AND EX/MEM.rd = ID/EX.rt:
```

```
    ForwardB = EX/MEM.ALUResult
```

```
else if MEM/WB.RegWrite AND MEM/WB.rd = ID/EX.rt:
```

```
    ForwardB = WB_WriteData
```

```
else:
```

```
    ForwardB = ID/EX.ReadData2
```

After forwarding, the ALU input A passes through the SVPC mux. This mux selects between the forwarded register value and the PC. For normal ALU instructions, ALU input A comes from R[rs]. For SVPC, ALU input A must be the PC because the instruction computes PC + immediate:

```
if ID/EX.SVPC:
```

```
    ALU_A = ID/EX.PC
```

```
else:
```

```
    ALU_A = ForwardA
```

ALU input B passes through the ALUSrc mux. For register-register operations, input B is the forwarded R[rt] value. For immediate operations such as INC and SVPC, input B is the sign-extended immediate:

```

if ID/EX.ALUSrc:
    ALU_B = ID/EX.Immediate
else:
    ALU_B = ForwardB

```

The ALU then performs the operation selected by ALUOp:

```

if ALUOp = ADD:
    ALUResult = ALU_A + ALU_B
else if ALUOp = SUB:
    ALUResult = ALU_A - ALU_B
else if ALUOp = NEG:
    ALUResult = -ALU_A
else if ALUOp = PASS_A:
    ALUResult = ALU_A
else:
    ALUResult = 0 or don't care

```

The ALU also generates the Zero and Negative flags:

ALU_Z = 1 if ALUResult = 0

ALU_N = 1 if ALUResult is negative

The EX stage passes the ALU result, stores write-data, destination register, flags, and control signals into the EX/MEM register. For store instructions, the data to be stored is the forwarded value from R[rt]:

4. Memory Stage

The Memory stage contains the data memory. The memory address comes from the ALU result stored in the EX/MEM pipeline register:

DataMemoryAddress = EX/MEM.ALUResult

For a store instruction, the data to be written is:

DataMemoryWriteData = EX/MEM.WriteData

The memory behavior is controlled by MemRead and MemWrite:

```

if EX/MEM.MemWrite:
    Mem[DataMemoryAddress] = DataMemoryWriteData

```

if EX/MEM.MemRead:

DataMemoryReadData = Mem[DataMemoryAddress]

For LD, the data memory output is passed to the writeback stage and eventually written into R[rd]. For ST, the value from R[rt] is written into data memory at address R[rs]. For JM, the data memory output becomes the PC redirect target, because JM is defined as:

PC = Mem[R[rs]]

This is why JM is resolved later than BRZ and BRN. Even though JM is decoded in ID, the new PC target cannot be known until the data memory output is available in the MEM stage. When EX/MEM.JM = 1, the CPU redirects the PC to the data memory output:

if EX/MEM.JM:

PC_next = DataMemoryReadData

Because this redirect happens late, instructions from the wrong path may already be in both IF/ID and ID/EX. Therefore, JM causes both of these pipeline registers to flush:

if EX/MEM.JM:

IF/ID_flush = 1

ID/EX_flush = 1

The MEM stage then passes the memory read-data, ALU result, destination register, and remaining writeback control signals into the MEM/WB pipeline register.

5. Writeback Stage

The Writeback stage writes the final result back into the register file. The value written back can come from either the ALU result or data memory. The MemtoReg control signal selects the writeback source:

if MEM/WB.MemtoReg:

WB_WriteData = MEM/WB.MemoryReadData

else:

WB_WriteData = MEM/WB.ALUResult

If RegWrite is enabled, the selected writeback data is written into the destination register:

if MEM/WB.RegWrite:

R[MEM/WB.rd] = WB_WriteData

Instructions such as ADD, SUB, NEG, INC, and SVPC write the ALU result back to the register file. The LD instruction writes memory data back to the register file. Instructions such as ST, JM, BRZ, BRN, and NOP do not write to the register file.

The writeback value is also used by the forwarding unit and branch-target forwarding logic. This allows later instructions to use the newest available value even if it has not yet been read normally from the register file.

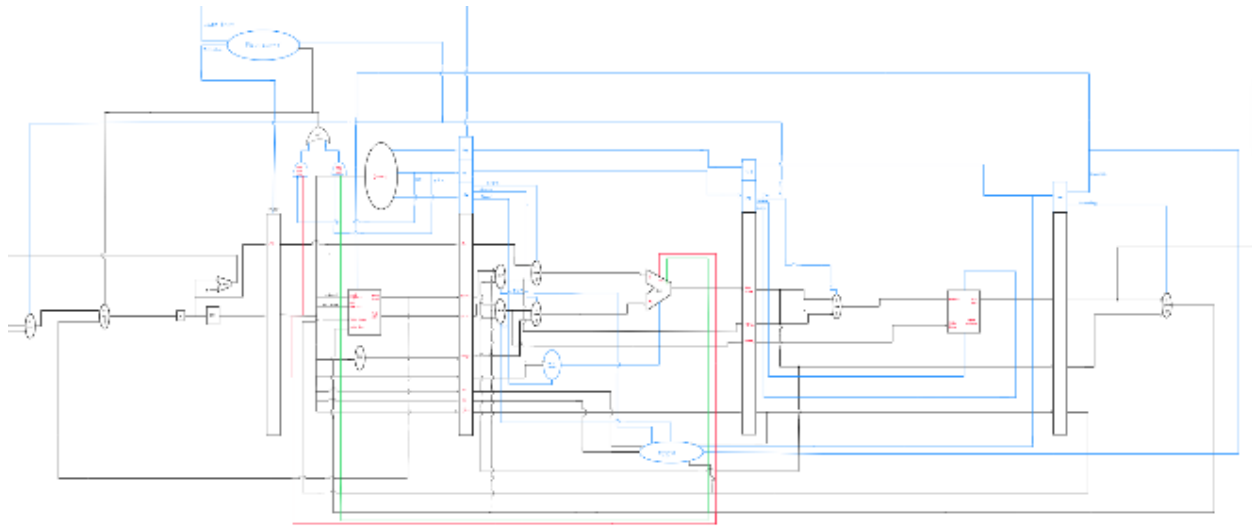


Figure 1. [Datapath](#)

IV. Control Unit

The Control Unit decodes the 4-bit instruction opcode into the following signals to help dictate functions and select muxes:

- BRN: Signal for branch if negative that is immediately used for ID branch logic.
- BRZ: Special branch if zero control bit, also used only in the ID stage for branching.
- ALUOp: Consists of 3 bits for the ADD, NEG, and SUB control inputs to the ALU.
- SVPC: Chooses PC over R[rs] as the first ALU operand within the pipeline's EX step.
- ALUSrc: Picks the immediate value from ID/EX registers over R[rt] for ALU input two.
- MemWrite: Enables writing to the data memory in MEM.
- MemRead: Triggers reading from the data memory in MEM.
- JM: A bit uniquely used in two phases of the pipeline, firstly to select R[rs] as the address used by DM, then used in the WB stage to set the value of PC to
- MemToReg: Selects the memory read output over the ALU output as the RF write data.
- RegWrite: Enables the wrt option in the RF during the WB phase.

Table 3. Control Unit Truth Table

Instruction	opcode	BRN	BRZ	ALUOp	SVPC	ALUSrc	MemWrite	MemRead	JM	MemToReg	RegWrite
NOP	0000	0	0	010	0	0	0	0	0	0	0
ST	0011	0	0	111	0	0	1	0	0	0	0
ADD	0100	0	0	000	0	0	0	0	0	0	1
INC	0101	0	0	000	0	1	0	0	0	0	1
NEG	0110	0	0	110	0	0	0	0	0	0	1
SUB	0111	0	0	101	0	0	0	0	0	0	1
JM	1000	0	0	111	0	0	0	1	1	0	0
BRZ	1001	0	1	111	0	0	0	0	0	0	0
BRN	1011	1	0	111	0	0	0	0	0	0	0
LD	1110	0	0	111	0	0	0	1	0	1	1
SVPC	1111	0	0	000	0	1	0	0	0	0	1

V. Verification Benchmark and Waveform

The CPU was verified using a Verilog testbench. The testbench first initialized instruction memory to NOPs, then loaded the translated demo program into specific instruction memory addresses. The benchmark was designed to test SVPC, INC, NEG, BRN, ADD, ST, LD, SUB, BRZ, JM, and NOP.

```
# =====
# Demo assembly for current working testbench
# =====

# Label addresses:
# LABEL1 = 15
# SKIP1 = 21
```

```

# LABEL2 = 31
# SKIP2 = 37

# Since SVPC does:
# xrd = PC + immediate
#
# These offsets are used:
# PC 1: x10 = 1 + 14 = 15
# PC 2: x11 = 2 + 29 = 31
# PC 3: x12 = 3 + 18 = 21
# PC 4: x13 = 4 + 33 = 37

0: SVPC x1, 100      # x1 = 100
1: SVPC x10, 14      # x10 = LABEL1 = 15
2: SVPC x11, 29      # x11 = LABEL2 = 31
3: SVPC x12, 18      # x12 = SKIP1 = 21
4: SVPC x13, 33      # x13 = SKIP2 = 37

5: NOP
6: NOP

7: INC x2, x1, 4      # x2 = x1 + 4 = 104
8: NEG x3, x1         # x3 = -x1 = -100

9: NOP
10: NOP

11: BRN x10           # branch to LABEL1 because previous Neg = 1
12: NOP
13: NOP
14: INC x2, x2, 17     # should not execute

LABEL1:
15: ADD x4, x1, x2     # x4 = 100 + 104 = 204

16: NOP
17: NOP

18: BRN x12           # should not branch because previous Neg = 0
19: NOP

```

```

20: INC x4, x4, 6    # x4 = 210

SKIP1:
21: ST x1, x1        # Mem[x1] = x1, so Mem[100] = 100
22: NOP

23: LD x5, x1        # x5 = Mem[x1] = Mem[100] = 100
24: NOP
25: NOP

26: ADD x6, x1, x2    # x6 = 100 + 104 = 204
27: SUB x7, x5, x1    # x7 = 100 - 100 = 0

28: NOP
29: NOP

30: BRZ x11          # branch to LABEL2 because previous Zero = 1

LABEL2:
31: ADD x8, x1, x2    # x8 = 100 + 104 = 204

32: NOP
33: NOP

34: BRZ x13          # should not branch because previous Zero = 0
35: NOP
36: INC x8, x8, 10    # x8 = 204 + 10 = 214

SKIP2:
37: INC x9, x1, -5    # x9 = 100 - 5 = 95
38: NOP
39: NOP

40: JM x1             # PC = Mem[x1] = Mem[100] = 100

# At instruction memory address 100:
100: JM x1            # keeps jumping to address stored in Mem[100]

```

The expected final values were:

```
x1 = 100
x2 = 104
x3 = -100
x4 = 210
x5 = 100
x6 = 204
x7 = 0
x8 = 214
x9 = 95
Mem[100] = 100
```

The simulation waveform confirmed these values (figure 2):

```
x1 = 00000064 = 100
x2 = 00000068 = 104
x3 = fffff9c = -100
x4 = 000000d2 = 210
x5 = 00000064 = 100
x6 = 000000cc = 204
x7 = 00000000 = 0
x8 = 000000d6 = 214
x9 = 0000005f = 95
Mem[100] = 00000064 = 100
```

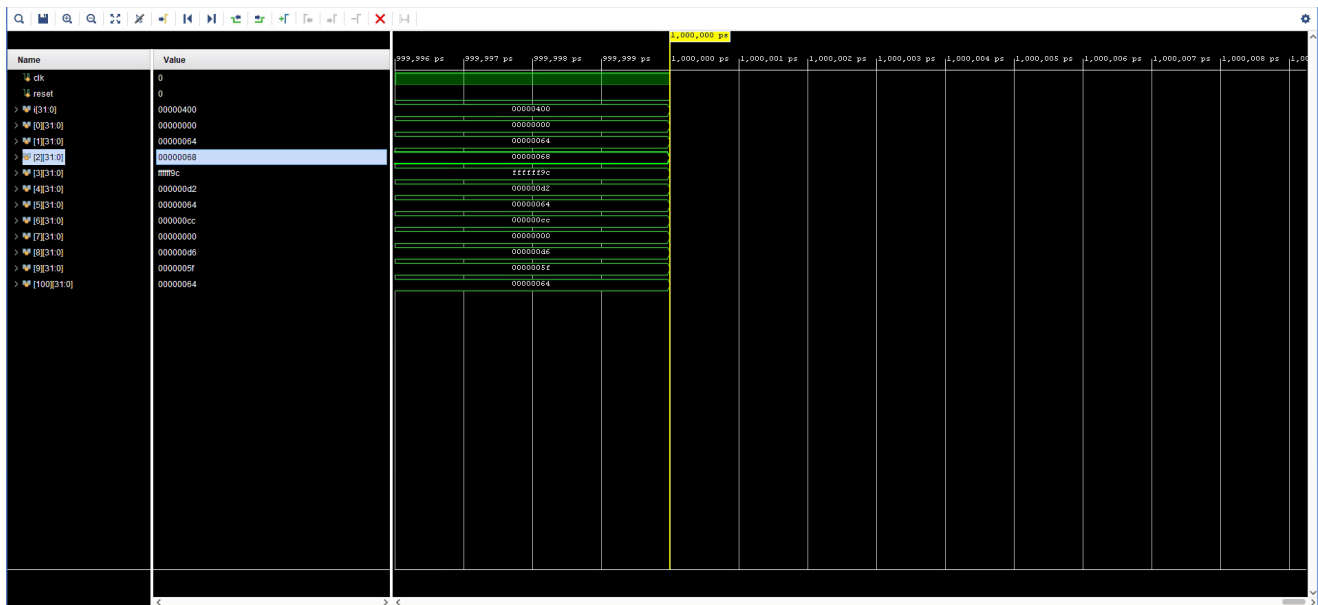


Figure 2. Final WaveForm

VI. Vector Addition Assembly Program

The required benchmark performs vector addition: $a[i] = a[i] + b[i]$ for $i = 0$ to $n - 1$. This version uses only the 11 required base instructions and does not use ADDV.

Assumptions: $x0 = 0$, $x1$ = base address of array a, $x2$ = base address of array b, $x3 = n$, $x4$ = LOOP address, $x5$ = DONE address, $x6 = a[i]$, $x7 = b[i]$, $x8$ = sum, $x9$ = loop counter, and $x10$ = temporary value for unconditional branching.

```
# =====
# Vector Addition without ADDV
#  $a[i] = a[i] + b[i]$ , for  $i = 0$  to  $n - 1$ 
# =====
#
# Assumptions:
#  $x0 = 0$ 
#  $x1$  = base address of array a
#  $x2$  = base address of array b
#  $x3 = n$ 
#  $x4$  = LOOP address
#  $x5$  = DONE address
#  $x6$  = temporary  $a[i]$ 
#  $x7$  = temporary  $b[i]$ 
#  $x8$  = sum
#  $x9$  = loop counter
#  $x10$  = temporary for unconditional branch
# =====

    ADD    x9, x3, x0      #  $x9 = n$ 

    SVPC   x4, LOOP_OFFSET #  $x4$  = LOOP address
    SVPC   x5, DONE_OFFSET  #  $x5$  = DONE address

LOOP:
    LD     x6, x1          #  $x6 = \text{Mem}[x1] = a[i]$ 
    NOP
    NOP
```

LD	x7, x2	# x7 = Mem[x2] = b[i]
NOP		
NOP		
ADD	x8, x6, x7	# x8 = a[i] + b[i]
NOP		
NOP		
ST	x8, x1	# Mem[x1] = x8
INC	x1, x1, 1	# x1 = x1 + 1, next a element
INC	x2, x2, 1	# x2 = x2 + 1, next b element
INC	x9, x9, -1	# x9 = x9 - 1
NOP		
NOP		
BRZ	x5	# if x9 == 0, branch to DONE
INC	x10, x0, 1	# x10 = 1
NEG	x10, x10	# x10 = -1, sets Negative flag
NOP		
NOP		
BRN	x4	# unconditional branch to LOOP
DONE:		
NOP		

VII. Performance Analysis

The CPU has five pipeline stages: IF, ID, EX, MEM, and WB. The project provides the following component delays: instruction memory delay is 2 ns, data memory delay is 2 ns, register file delay is 1.5 ns, and ALU delay is 2 ns. Other component delays are ignored.

The estimated stage delays are:

Stage	Main Component	Delay
IF	Instruction Memory	2 ns
ID	Register File	1.5 ns

EX	ALU	2 ns
MEM	Data Memory	2 ns
WB	Register File Write	1.5 ns

The clock period is determined by the slowest pipeline stage:

$T_{clk} = \max(2ns, 1.5ns, 2ns, 2ns, 1.5ns)$

$T_{clk} = 2ns$

Therefore, the maximum clock frequency is: $1/2ns = 500 \text{ MHz}$

1. Demo/Testbench Program Performance Analysis

The demo waveform was generated from the fixed testbench program, not the vector-add benchmark. This program verifies that the implemented CPU correctly executes the required instructions and handles pipeline control behavior. The main demo program occupies instruction memory addresses 0 through 40, with an additional JM x1 instruction placed at address 100.

The useful execution path includes the first taken BRN, the non-taken BRN, the taken BRZ, the non-taken BRZ, and the final JM. The approximate useful dynamic instruction count before reaching the repeating JM x1 at address 100 is 39 instructions. A five-stage pipeline has an approximate fill cost of 4 cycles. The main redirect penalties before the repeating jump are approximately 1 cycle for the taken BRN at address 11, 1 cycle for the taken BRZ at address 30, and 2 cycles for the JM at address 40.

Demo cycles $\approx 39 + 4 + 4 = 47$ cycles

Demo CPI $\approx 47 / 39 \approx 1.21$

The CPI is greater than 1 because the demo program includes inserted NOPs and control-flow flushes. The purpose of this benchmark is functional verification rather than optimal performance.

2. Vector-Addition Benchmark Performance

The vector-add benchmark is the scalable benchmark because its instruction count and cycle count depend on problem size n . The setup portion contains three instructions. Each non-final loop iteration takes approximately 21 instructions, while the final iteration takes approximately 16 instructions because the unconditional branch sequence back to LOOP is not executed.

Instruction Count = $3 + 21(n - 1) + 16$

$$\text{Instruction Count} = 21n - 2$$

A five-stage pipeline requires approximately four fill cycles. The loop-back BRN is taken for $n - 1$ iterations, and the final BRZ is taken once. Therefore, the number of taken control transfers is approximately n . Using a conservative one-cycle penalty per taken branch gives:

$$\text{Total Cycles} \approx \text{Instruction Count} + \text{Pipeline Fill} + \text{Branch Penalty}$$

$$\text{Total Cycles} \approx (21n - 2) + 4 + n$$

$$\text{Total Cycles} \approx 22n + 2$$

The estimated CPI is:

$$\text{CPI} \approx \text{Total Cycles} / \text{Instruction Count}$$

$$\text{CPI} \approx (22n + 2) / (21n - 2)$$

$$\text{For large } n: \text{CPI} \approx 22/21 \approx 1.05$$

This analysis does not use ADDV because ADDV was not implemented in the CPU.